



AUDIT REPORT

May 2026

For



BULLBIT

Table of Content

Executive Summary	04
Number of Security Issues per Severity	06
Summary of Issues	07
Checked Vulnerabilities	09
Techniques and Methods	11
Types of Severity	12
Types of Issues	14
Severity Matrix	15
 Low Severity Issues	16
1. Cross-contract storage reads in validateUserOp violate ERC-4337 v0.7 simulation rules (ERC-7562)	16
2. Recovery flows leave EOA backup AND guardian set live	17
3. Missing __gap arrays on UUPS contracts	18
4. Missing Chain ID in Backup Recovery Signed Message	19
5. Excessively High Per-Transaction Gas Cost Cap	20
6. Redundant State and Unused Inheritance Increase Gas Costs	21
7. Paymaster Sponsors Arbitrary Calls for Counterfactual Wallets Without Target Whitelist	22
 Informational Issues	23
8. addEOAOwner Silently Replaces Prior Owner	23
9. Unbounded array iteration can cause gas-heavy token removal where removeApprovedToken()	24
10. Missing Session Key Lifecycle Validation	25
11. Missing Existence Check Before Revoking Session Key	26



12. Missing Validation for Initializer Addresses	27
Functional Tests	28
Automated Tests	28
Threat Model	29
Closing Summary & Disclaimer	56



Executive Summary

Project Name	Bullbit
Protocol Type	Account Abstraction Wallet (ERC-4337)
Project URL	https://bullbit.ai/
Overview	Bullbit is an account abstraction-based wallet system built on ERC-4337. The protocol consists of upgradeable smart wallets, a deterministic deployment factory, and a paymaster for gas sponsorship. Users authenticate using WebAuthn-based passkeys, with additional support for backup EOA keys and guardian-based social recovery mechanisms. The WalletFactory deploys wallet proxies using CREATE2, enabling deterministic wallet addresses and centralized configuration such as auto-sweep settings and default parameters. The SmartWallet contract manages account ownership, transaction execution, session keys, delegated permissions, and recovery flows. The Paymaster contract enables selective gas sponsorship for user operations, enforcing policy checks and whitelisting to control costs and usage.
Audit Scope	The scope of this Audit was to analyze the Bullbit Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/bullbit/Smart-Wallet-Contract/tree/main
Contracts in Scope	Paymaster.sol SimpleEntryPoint.sol SmartWallet.sol WalletFactory.sol Base64URL.sol CustomErrors.sol PackingHelper.sol WebAuthnP256.sol
Branch	main
Commit Hash	20c1614f39a663ef51e44e895892dd8a0e0abe05
Language	Solidity



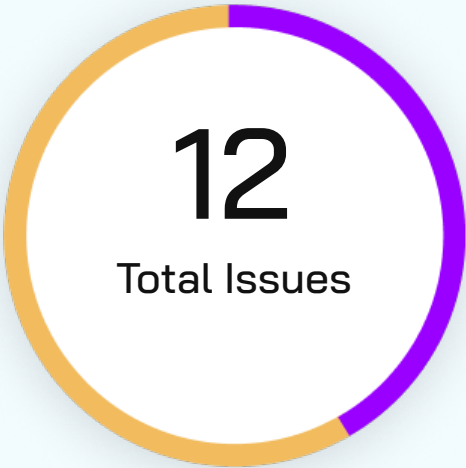
Blockchain	Base
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	4th April 2026 - 19th April 2026
Updated Code Received	1st May 2026 and 4th May 2026
Review 2	1st May and 5th May 2026
Fixed In	74f31877d4ea03763d00bc0af42c72d79c8fa9e3

Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>



Number of Issues per Severity



Critical	0 (0.0%)
High	0 (0.0%)
Medium	0 (0.0%)
Low	7 (58.3%)
Informational	5 (42.7%)

		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	0	1	0
	Partially Resolved	0	0	0	0	0
	Resolved	0	0	0	6	5



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Cross-Contract Storage Reads in validateUserOp Violate ERC-4337 v0.7 Simulation Rules (ERC-7562)	Low	Acknowledged
2	Recovery Flows Leave EOA Backup and Guardian Set Active	Low	Resolved
3	Missing __gap Arrays on UUPS Contracts	Low	Resolved
4	Missing Chain ID in Backup Recovery Signed Message	Low	Resolved
5	Excessively High Per-Transaction Gas Cost Cap	Low	Resolved
6	Redundant State and Unused Inheritance Increase Gas Costs	Low	Resolved
7	Paymaster Sponsors Arbitrary Calls for Counterfactual Wallets Without Target Whitelist	Low	Resolved
8	addEOAOwner Silently Replaces Prior Owner	Informational	Resolved
9	Unbounded Array Iteration Can Cause Gas-Heavy Token Removal in removeApprovedToken()	Informational	Resolved
10	Missing Session Key Lifecycle Validation	Informational	Resolved



Issue No.	Issue Title	Severity	Status
11	Missing Existence Check Before Revoking Session Key	Informational	Resolved
12	Missing Validation for Initializer Addresses	Informational	Resolved



Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations
Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls



✓ **Missing Zero Address Validation**

✓ **Private modifier**

✓ **Revert/require functions**

✓ **Multiple Sends**

✓ **Using suicide**

✓ **Using delegatecall**

✓ **Upgradeable safety**

✓ **Using throw**

✓ **Using inline assembly**

✓ **Style guide violation**

✓ **Unsafe type inference**

✓ **Implicit visibility level**

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.



Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.



Severity Matrix

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



Low Severity Issues

Cross-contract storage reads in validateUserOp violate ERC-4337 v0.7 simulation rules (ERC-7562)

Acknowledged

Description

The production deployment target is the canonical ERC-4337 EntryPoint v0.7 (SimpleEntryPoint exists only as a local test fixture and is out of scope). On the canonical EntryPoint, every compliant bundler (alto, voltaire, skandha, stackup) enforces the ERC-7562 validation-scope rules during off-chain simulation before accepting a UserOp into the mempool.

validateUserOp→_validateSignature→_validateEOASignatureWithPermission reads four slots from an external, non-staked contract (WalletFactory) inside the account validation phase:

```
factory.getDefaultAutoSweepKey()  
factory.getDefaultPoolContract()  
factory.getDepositSelector()  
factory.isTokenApproved(target)
```

None of these qualify as “associated storage” of the account under ERC-7562 Storage Rules: the first three are single global slots, and isTokenApproved is a mapping(address => bool) keyed by the target token, not by the wallet address.

Under ERC-7562, the only storage accesses permitted during account validation for an unstaked account are:

STO-010 — account’s own storage (not satisfied — reads are external)

STO-021 / STO-022 — associated storage of the account in a non-entity contract (not satisfied — reads aren’t keyed by address(this))

STO-033 — any storage in a non-entity contract, but only when a staked entity is in play (not satisfied — WalletFactory is never staked: addStake is not called anywhere, and the factory is not an entity of the UserOp in post-deployment flows)

No permissive rule applies, so the reads are forbidden by default and will be rejected by any compliant bundler’s simulation tracer.

Impact

Every Auto-Sweep UserOp is dropped at eth_sendUserOperation simulation by compliant bundlers. Auto-Sweep is non-functional on production bundler infrastructure. Protocol’s feature breakage.

Likelihood

Low

Recommendation

Stake WalletFactory with the EntryPoint, OR cache the relevant Factory config inside the wallet at registration time and re-fetch only outside the validation phase, OR move Auto-Sweep validation into the execution phase entirely.



Recovery flows leave EOA backup AND guardian set live

Resolved

Description

Both recovery functions invoke `_revokeAllPasskeys()` (clears passkeys) but neither clears the OTHER recovery mechanism's keys.

After `executeSocialRecovery`: `eoasOwners[oldEOA]` remains true → `_isOwner` still returns true for old EOA → old EOA retains full `execute/executeBatch` access.

After `recoverWithBackup`: guardians, `guardianList`, `guardianThreshold` remain unchanged guardian quorum can immediately undo via `executeSocialRecovery`.

This creates a permanent counter-recovery race: legitimate user does social recovery, attacker (still holding compromised EOA backup) immediately calls `recoverWithBackup` to undo, and so on.

Attacker wins via mempool monitoring + front-running.

Impact

Medium , direct fund loss

Likelihood

Low

Recommendation

Each recovery function MUST clear ALL alternative authorization paths atomically: `executeSocialRecovery` should clear `eoasOwners/eoaOwnerList` (or require explicit re-add), and `recoverWithBackup` should clear `guardians/guardianList/guardianThreshold`. Optionally: provide a separate `recoveryWithReset(...)` that clears everything explicitly.



Missing __gap arrays on UUPS contracts

Resolved

Description

All three UUPS-upgradeable contracts have NO __gap arrays. While OZ v5 isolates parent state into ERC-7201 namespaces (so v5→v5 parent collision is unlikely), absence of gaps creates forward-compatibility hazards: future intermediate base contracts shifting child slots, field reordering during V2 refactor with no safety net.

Impact

Possibility of storage collision

Likelihood

Low to Medium

Recommendation

Add uint256[50] private __gap; to each contract;



Missing Chain ID in Backup Recovery Signed Message

Resolved

Path

SmartWallet.sol

Function Name

`recoverWithBackup`

Description

The backup recovery message hash does not include block.chainid. It is currently built from "BULLBIT_BACKUP_RECOVERY", address(this), the new passkey values, and walletNonce only (src/SmartWallet.sol (line 1028)). If the same wallet address exists on another chain and the nonce is aligned, a backup signature collected for one chain can be replayed on that other chain.

Impact

Medium

Likelihood

Low

Recommendation

Include block.chainid in the signed recovery payload



Excessively High Per-Transaction Gas Cost Cap

Resolved

Path

Paymaster.sol

Function Name

`validatePaymasterUserOp`

Description

The paymaster accepts any sponsored operation whose `maxCost` is up to `MAX_GAS_COST = 1 ether` (src/ Paymaster.sol (line 29)). This cap is extremely high for a single `UserOperation` and does not appear aligned with the contract's anti-spam objective. As a result, a single whitelisted operation can consume a disproportionately large portion of the paymaster's ETH balance.

Impact

Medium

Likelihood

Low

Recommendation

Lower `MAX_GAS_COST` to a realistic ceiling derived from expected operation types and supported networks



Redundant State and Unused Inheritance Increase Gas Costs

Resolved

Path

SmartWallet.sol

Function Name

`initialize, cancelRecovery`

Description

The contract includes EIP712Upgradeable inheritance and calls `__EIP712_init("SmartWallet", "1")` during initialization even though the current implementation does not appear to use any EIP-712 typed-data workflow. This adds unnecessary bytecode, initialization logic, and inherited storage complexity without improving functionality. Keeping unused inheritance in a wallet contract also makes the codebase harder to maintain because it suggests a signing model that is not actually enforced anywhere in the active flow.

The contract defines a dedicated `withdrawNonce`, but that variable is never read, incremented, or used to protect any withdrawal-specific operation. As a result, the project pays for an extra storage slot and additional conceptual complexity without receiving any replay-protection or execution benefit from it. Unused state variables are especially undesirable in upgradeable systems because storage layout should stay as lean and intentional as possible.

The `cancelRecovery` function clears the entire `pendingRecovery` struct even though disabling the recovery flow only requires invalidating its active flag. Wiping every field in the struct causes additional storage writes and therefore increases gas consumption on a path that could be implemented more cheaply. While this is not a security bug, it is still an avoidable inefficiency that slightly worsens user cost and contract execution overhead.

Impact

Medium

Likelihood

Low

Recommendation

Remove the unused EIP712Upgradeable inheritance unless typed-data signing is actually needed, eliminate the unused `withdrawNonce` storage variable, and update recovery cancellation logic to perform only the minimum storage changes required to disable the pending recovery.



Paymaster Sponsors Arbitrary Calls for Counterfactual Wallets Without Target Whitelist

Resolved

Path

Paymaster.sol

Function Name

`validatePaymasterUserOp()`

Description

When `userOp.initCode.length > 0`, the Paymaster enters the counterfactual wallet path and calls `_isEssentialOperation(opSelector)`. This function returns true for `execute(address,uint256,bytes)` and `executeBatch(address[],uint256[],bytes[])`, causing an immediate approval return. This early return bypasses the target whitelist enforcement that is applied to all already-deployed wallets. Compounding this, the Paymaster performs no validation on the factory address embedded in `initCode[0:20]`. An attacker can supply an entirely attacker-controlled factory that deploys a malicious wallet whose `validateUserOp` unconditionally returns 0 (valid) regardless of the signature provided. Because the wallet passes account validation and the Paymaster's counterfactual path approves `execute` without checking targets, the `EntryPoint` executes the operation and charges the full gas cost to the Paymaster's `EntryPoint` deposit.

Impact

Medium

An attacker can repeatedly submit `UserOps`, each deploying a new attacker-controlled wallet via a fresh salt, and each calling `execute` on any arbitrary target at Paymaster expense.

Likelihood

Low

Recommendation

If `execute/executeBatch` must be sponsored during deployment, route them through the same target whitelist enforcement as deployed wallets. Add factory address validation in `initCode`.



Informational Issues

addEOAOwner Silently Replaces Prior Owner

Resolved

Description

Function name suggests addition; implementation deletes every existing EOA owner before adding the new one (enforces MAX_EOA_OWNERS = 1 implicitly). User intending to add a co-signer silently revokes their existing backup key. If they later lose the new key, no backup exists.

Recommendation

Rename to setEOAOwner or replaceEOAOwner; emit a replacement-prominent event; explicit NatSpec; and consider raising MAX_EOA_OWNERS to support real backup configurations.



Unbounded array iteration can cause gas-heavy token removal where `removeApprovedToken()`

Resolved

Path

WalletFactory.sol

Function Name

`removeApprovedToken`

Description

The `removeApprovedToken` function removes entries from `approvedTokens` by linearly scanning the full array until it finds the target token, then performing a swap-and-pop. Because no index mapping is maintained, the gas cost of removal grows with the size of the approved token list. As the list expands, administrative token removals become increasingly expensive and may eventually become operationally cumbersome.

Impact

Low

Likelihood

Low

Recommendation

Store each approved token's array index in a mapping so removals can be performed in constant time with swap-and-pop, without iterating across the entire array.



Missing Session Key Lifecycle Validation

Resolved

Path

SmartWallet.so

Function Name

`registerSessionKey`, `revokeSessionKey`

Description

The session key lifecycle logic lacks important validation at both creation and revocation time. In `registerSessionKey`, the contract checks only that `validUntil` is greater than the current timestamp, but it does not verify that `validAfter` is less than `validUntil`, which allows invalid activation windows to be stored. In `revokeSessionKey`, the contract marks a session key as inactive without first confirming that the key was ever registered or is currently active, which can produce misleading state transitions and events. Together, these gaps weaken the correctness and reliability of session key administration.

Impact

Low

Likelihood

Low

Recommendation

Require `validAfter < validUntil` during session key registration



Missing Existence Check Before Revoking Session Key

Resolved

Path

SmartWallet.so

Function Name

`revokeSessionKey`

Description

The `revokeSessionKey` function unconditionally sets `sessionKeys[sessionKey].isActive = false` without first checking whether the session key was ever registered or is currently active. This means the contract can emit a successful revocation event for a nonexistent or already inactive key, creating misleading state transitions and making off-chain monitoring or administrative tooling less reliable

Recommendation

Validate that the session key exists and is currently active before revoking it, and revert otherwise.



Missing Validation for Initializer Addresses

Resolved

Path

src/WalletFactory.sol, src/Paymaster.sol, src/SmartWallet.sol

Function Name

`initialize`

Description

The initializer functions assign critical addresses such as `entryPoint`, `walletImplementation`, and owner-related configuration without consistently validating that the supplied values are nonzero and valid. If an incorrect address is provided during deployment or upgrade setup, the contract can be initialized into a broken or unusable state with no straightforward recovery path.

Recommendation

Validate all critical initializer parameters before storing them, especially `entryPoint`, implementation addresses, and owner addresses, and revert on zero or otherwise invalid values.



Functional Tests

Some of the tests performed are mentioned below:

- ✓ WalletFactory initialization and owner-controlled configuration updates
- ✓ WalletFactory deterministic wallet deployment and wallet address prediction
- ✓ SmartWallet initialization, owner setup, and execution flow
- ✓ SmartWallet passkey authentication and signature validation
- ✓ SmartWallet EOA backup owner management
- ✓ SmartWallet session key registration, usage, and revocation
- ✓ SmartWallet guardian configuration and social recovery flow
- ✓ SmartWallet backup recovery with signed authorization
- ✓ SmartWallet permission management for delegated signers and auto-sweep controls
- ✓ Paymaster initialization, selector whitelisting, and sponsored transaction validation
- ✓ Paymaster deposit and withdrawal mechanisms
- ✓ SimpleEntryPoint UserOperation handling and wallet/paymaster interaction
- ✓ WebAuthnP256 signature verification logic
- ✓ PackingHelper packing and unpacking utilities
- ✓ Base64URL encoding and decoding helper behavior
- ✓ CustomErrors revert conditions across invalid flows
- ✓ Mock contract behavior such as reentrancy and edge-case execution paths

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.



Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Canonical ERC-4337 EntryPoint v0.7	External Contract	Only authorized caller of SmartWallet.validateUserOp, Paymaster.validatePaymasterUserOp, and Paymaster.postOp. Custody sink for Paymaster gas deposits via depositTo / withdrawTo	Canonical EntryPoint is correctly deployed, non-upgradeable, and behaves per EIP-4337 spec	If an incorrect address is set as entryPoint in SmartWallet/ Paymaster initializers, all validation flows break; EntryPoint bugs propagate to every wallet
ERC-4337 Bundlers	Off-chain Actor	Simulate UserOps against ERC-7562 rules before inclusion; front-run-unaware propagation	Compliant bundlers enforce ERC-7562 storage/ opcode rules in validation phase	UserOps that violate ERC-7562 are dropped at mempool admission – e.g. Auto-Sweep path reads WalletFactory globals during validation
OpenZeppelin Initializable	Library	Inherited by SmartWallet, WalletFactory, Paymaster to gate initialize	Initializer modifier correctly enforces one-time initialization	N/A



Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
UUPSUpgradeable + ERC1967Proxy	Library/Proxy	Proxy pattern for SmartWallet / WalletFactory / Paymaster; _authorizeUpgrade gates implementation swap	Proxy slot layout matches ERC-1967; upgradeToAndCall only callable by authorized upgrader	Unauthorized upgrade = full contract takeover; storage-layout drift between implementations corrupts state
Ownable2StepUpgradeable	Library	Owner access control for WalletFactory and Paymaster; two-step ownership handover	acceptOwnership guards against address typos; owner is trusted operator key	Owner compromise = factory upgrade hijack, Paymaster drain, malicious origin/sweep-key injection into new wallets
EIP712Upgradeable	Library	Initialized in SmartWallet for domain separator (unused in current call paths but initialized)	Domain separator matches EIP-712	N/A
ECDSA + MessageHashUtils	Library	EOA backup / permission / recovery signatures use toEthSignedMessageHash().tryRecover()	Malleability-rejecting tryRecover variant; ErrorNoError is the only accepted outcome	N/A



Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Create2	Library	Deterministic proxy address computation in <code>WalletFactory.getWalletAddress</code>	Create2 deploys to the computed address	Salt collision = wallet address collision; only owner-created salts are deployable, so collisions require owner error
Base64URL	Library	Base64URL-encodes WebAuthn challenge; substring-matches JSON fields of <code>clientDataJSON</code>	Precompile call signature and fallback implementation both correct	Fallback divergence from precompile produces chain-specific validation discrepancies
WebAuthnP256	Library	Dispatches to RIP-7212 precompile with Solidity fallback	Operated by the protocol team; private key kept warm in relayer infrastructure	Compromise = attacker can approve unlimited ERC-20 allowance from every wallet to the configured pool, then drain via any subsequent pool interaction; bounded by "approved tokens" + "pool address" configured in Factory

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

SmartWallet.sol – per-user ERC-4337 account (ERC1967Proxy → UUPS implementation)

Contract Name	Function	Category
SmartWallet.sol	initialize(entryPoint, credentialId, pubKeyX, pubKeyY, emailHash, autoSweepEnabled, expectedOrigin)	What this function can do Bind the proxy to an EntryPoint, install the first passkey owner, record email hash, set auto-sweep opt-in, set expected origin, capture msg.sender as the trusted Factory What this function cannot/should not do Run twice, be called on a proxy whose passkeyCredentialIds is already non-empty, accept address(0) EntryPoint Main invariant(s) Exactly one successful initialize per proxy; factory = msg.sender is the immutable source of Auto-Sweep config; first passkey is always the bootstrap key Access level initializer modifier – first caller wins; expected caller is WalletFactory during createWallet*

Contract Name	Function	Category
SmartWallet.sol	validateUserOp(userOp, userOpHash, missingAccountFunds)	What this function can do Pay missingAccountFunds to EntryPoint; dispatch to passkey / EOA / session-key validators; return 0 (valid) or 1 (invalid) What this function cannot/should not do Read storage outside the account in a way that violates ERC-7562 (currently the Auto-Sweep path reads Factory globals – known ERC-7562 issue), revert instead of returning 1, approve an operation whose signature is malformed Main invariant(s) Only EntryPoint can invoke; validation must be deterministic, time-independent, and free of BLOCKED opcodes Access level onlyEntryPoint
SmartWallet.sol	execute(target, value, data)	What this function can do Forward arbitrary call + ETH to any non-zero target What this function cannot/should not do Execute when called by an address that is neither EntryPoint nor a current _isOwner, re-enter itself while a prior execute is in flight Main invariant(s) onlyOwnerOrEntryPoint is the sole gatekeeper of wallet funds; nonReentrant must hold across the call chain Access level onlyOwnerOrEntryPoint nonReentrant



Contract Name	Function	Category
SmartWallet.sol	executeBatch (targets[], values[], datas[])	What this function can do Forward an ordered batch of calls + ETH to targets What this function cannot/should not do Partial-commit (any failure reverts the batch), execute on length-mismatched arrays Main invariant(s) Atomic; same gatekeeper as execute Access level onlyOwnerOrEntryPoint nonReentrant
SmartWallet.sol	receive()	What this function can do Accept unsolicited ETH transfers into the wallet balance What this function cannot/should not do Execute logic, reject valid transfers Main invariant(s) Wallet ETH is owner-custodied from the moment it lands Access level Public payable
SmartWallet.sol	addPasskeyOwner(credentialId, pubKeyX, pubKeyY)	What this function can do Install a new passkey owner What this function cannot/should not do Exceed MAX_PASSKEY_OWNERS=100, re-install an existing active credential Main invariant(s) Every active entry in passkeyOwners is present in passkeyCredentialIds Access level onlySelf (only reachable via execute/executeBatch signed by current owner/EntryPoint)



Contract Name	Function	Category
SmartWallet.sol	removePasskeyOwner(credentialId)	What this function can do Deactivate a passkey and swap-pop its entry from the ID list What this function cannot/should not do Remove the last remaining owner (lockout guard) Main invariant(s) passkeyCredentialIds.length >= 1 after call Access level onlySelf
SmartWallet.sol	addEOAOwner(owner)	What this function can do Replace the entire eoaOwners set with a single new owner (enforced by the loop that clears the existing list) What this function cannot/should not do Set address(0), stack multiple EOA owners (wallet enforces MAX_EOA_OWNERS=1 implicitly via this replacement loop) Main invariant(s) eoaOwners holds at most one active address Access level onlySelf
SmartWallet.sol	removeEOAOwner(owner)	What this function can do Deactivate the EOA owner entry What this function cannot/should not do Leave the wallet ownerless – blocked if eoaOwnerList.length <= 1 && passkeyCredentialIds.length == 0 Main invariant(s) Removal cannot produce a no-owner state Access level onlySelf



Contract Name	Function	Category
SmartWallet.sol	addPermission(signer, target, selectors[])	What this function can do Grant a scoped per-(signer,target,selector) permission triple What this function cannot/should not do Grant permission to address(0) signer or target, accept empty selector list Main invariant(s) hasPermission[signer] implies signer ∈ permissionedSigners Access level onlySelf
SmartWallet.sol	removePermission(signer)	What this function can do Wipe the signer from hasPermission and permissionedSigners What this function cannot/should not do Remove an unknown signer Main invariant(s) Consistency between the boolean map and the array Access level onlySelf
SmartWallet.sol	setAutoSweepDisabled(disabled)	What this function can do Toggle the user-opt-out flag that short-circuits the Auto-Sweep branch of _validateEOASignatureWithPermission What this function cannot/should not do Affect any other wallet's configuration Main invariant(s) Flag is monotonic only under user action; not cleared by recovery flows Access level onlySelf



Contract Name	Function	Category
SmartWallet.sol	setExpectedOrigin(origin)	What this function can do Update the per-wallet expected WebAuthn origin used by _verifyWebAuthn What this function cannot/should not do Currently does not reject empty string – re-opens the origin-empty cohort if called with "" Main invariant(s) Non-empty origin enables strict origin matching; empty disables the check Access level onlySelf
SmartWallet.sol	registerSessionKey(sessionKey, validUntil, validAfter, maxVolume)	What this function can do Register a time-bounded, volume-bounded session key pinned to the current sessionNonce What this function cannot/should not do Register an already-expired session, register address(0) Main invariant(s) session.registeredAt == sessionNonce is required for activation; bumping sessionNonce via revokeAllSessions invalidates all prior sessions Access level onlySelf



Contract Name	Function	Category
SmartWallet.sol	revokeSessionKey(sessionKey)	What this function can do Deactivate a specific session by setting isActive = false What this function cannot/should not do Retroactively invalidate already-processed UserOps Main invariant(s) Idempotent on unknown keys Access level onlySelf
SmartWallet.sol	revokeAllSessions()	What this function can do Bump sessionNonce, invalidating all sessions whose registeredAt was the previous nonce Main invariant(s) sessionNonce monotonically increases; never decrements Access level onlySelf
SmartWallet.sol	updateSessionVolume(sessionKey, volumeUsed)	What this function can do Record cumulative volume used by a session key What this function cannot/should not do Decrement usage Access level onlySelf



Contract Name	Function	Category
SmartWallet.sol	setGuardians(guardians(), threshold)	What this function can do Replace the guardian set and set guardianThreshold; bumps recoveryNonce What this function cannot/should not do Produce guardians.length < 2 or > 5, or threshold == 0 or threshold > guardians.length; include address(0) or duplicates Main invariant(s) guardianThreshold <= guardianList.length Access level onlySelf
SmartWallet.sol	removeGuardians()	What this function can do Clear the guardian set, set guardianThreshold = 0, bump recoveryNonce Main invariant(s) After call, executeSocialRecovery is unreachable until guardians are re-set Access level onlySelf
SmartWallet.sol	cancelRecovery()	What this function can do Delete the pendingRecovery struct Main invariant(s) Only reachable by current owner; no timer required Access level onlySelf



Contract Name	Function	Category
SmartWallet.sol	cancelSocialRecovery()	What this function can do Bump recoveryNonce, invalidating all previously-signed guardian payloads Main invariant(s) Monotonic nonce Access level onlySelf
SmartWallet.sol	executeSocialRecovery(newCredentialId, newPubKeyX, newPubKeyY, signatures[])	What this function can do Revoke all passkeys and install newCredentialId if at least guardianThreshold distinct guardian signatures over the recovery hash are presented What this function cannot/should not do Accept duplicate signer, accept signatures from non-guardian addresses, reuse signatures across nonces Main invariant(s) Each successful call increments recoveryNonce Known design gap does NOT clear eoaOwners or permissions – compromised EOA retains execute / _isOwner access post-recovery Access level Permissionless (gated by signatures only)



Contract Name	Function	Category
SmartWallet.sol	recoverWithBackup(newCredentialId, newPubKeyX, newPubKeyY, backupSignature)	What this function can do Revoke all passkeys and install a new passkey using a valid EOA-backup signature over the recovery hash What this function cannot/should not do Accept a signature from a non-owner address, reuse a signature (nonce is walletNonce, incremented before verification) Main invariant(s) walletNonce is monotonic Access level Permissionless (gated by backup signature)
SmartWallet.sol	isValidSignature(hash, signature)	What this function can do EIP-1271 signature validator for off-chain contexts; supports EOA and passkey paths Main invariant(s) Return the magic value for invalid signatures Access level Public view
SmartWallet.sol	upgradeToAndCall(newImpl, data) (via UUPSUpgradeable)	What this function can do Swap the wallet implementation Main invariant(s) layout of newImpl must be strictly compatible Access level onlySelf (via _authorizeUpgrade)



Contract Name	Function	Category
SmartWallet.sol	View getters (getPasskeyOwners, getEOAOwners, getGuardians, getPendingRecovery, getPermissionedSigners, checkPermission, getAutoSweepConfig, getSessionKeyInfo, hasRemainingVolume)	What this function can do Read-only introspection Access level Public view



WalletFactory.sol — deterministic factory + centralized Auto-Sweep / origin configuration (owner-controlled, UUPS)

Contract Name	Function	Category
WalletFactory.sol	initialize(entryPoint, walletImplementation, owner)	What this function can do Bind the Factory proxy to an EntryPoint, wallet implementation, and owner What this function cannot/should not do Run twice; set empty defaults for defaultExpectedOrigin is currently permitted (the origin defaults to "" and wallets created before the owner sets it inherit this — a known default-cohort concern) Main invariant(s) successful initialization per proxy Access level initializer
WalletFactory.sol	createWallet(credentialId, pubKeyX, pubKeyY, salt, emailHash)	What this function can do Deploy an ERC1967Proxy pointing at walletImplementation, initialize the wallet with _autoSweepEnabled = false and the Factory's current defaultExpectedOrigin What this function cannot/should not do Produce two wallets for the same emailHash, deploy with emailHash == 0 Main invariant(s) emailToWallet[emailHash] and credentialToWallet[credentialId] are one-to-one with deployed wallets Access level onlyOwner nonReentrant



Contract Name	Function	Category
WalletFactory.sol	createWallet WithAutoSweep(credentialId, pubKeyX, pubKeyY, salt, emailHash)	What this function can do Same as createWallet but sets _autoSweepEnabled = true, requiring defaultAutoSweepKey and defaultPoolContract to be pre-configured What this function cannot/should not do Deploy when Auto-Sweep config is missing Main invariant(s) Every Auto-Sweep-enabled wallet inherits the same sweep key / pool / approved-token set live-bound to the Factory Access level onlyOwner nonReentrant
WalletFactory.sol	getWalletAddress(credentialId, pubKeyX, pubKeyY, salt)	What this function can do Pure CREATE2 address prediction for counterfactual quoting Public view Access level
WalletFactory.sol	getInitCode(...) / getInitCodeWithAutoSweep(...)	What this function can do Build the initCode blob for lazy deployment via UserOp.initCode Access level Public view
WalletFactory.sol	checkEmail(emailHash) / getWalletByCredential(credentialId) / isWalletDeployed(wallet)	What this function can do Read-only lookups Public view Access level



Contract Name	Function	Category
WalletFactory.sol	setWalletImplementation(newImpl)	What this function can do Change the implementation address new wallets will point to (existing wallets are unaffected, because each wallet's proxy captures its implementation at deployment time) What this function cannot/should not do Point at address(0) Main invariant(s) Only affects wallets created AFTER the call Access level onlyOwner
WalletFactory.sol	setDefaultAutoSweepKey(key)	What this function can do Hot-rotate the Auto-Sweep signing key read by every non-opted-out wallet What this function cannot/should not do Set address(0) Main invariant(s) Effective immediately for all wallets; changes are live-bound, not captured Access level onlyOwner
WalletFactory.sol	setDefaultExpectedOrigin(origin)	What this function can do Change the WebAuthn origin defaults for newly-created wallets What this function cannot/should not do Currently allows empty string – writing "" re-opens the origin-empty default cohort for any wallet minted afterwards Access level onlyOwner



Contract Name	Function	Category
WalletFactory.sol	setDepositSelector(selector)	What this function can do Hot-rotate the deposit(...) selector that Auto-Sweep signers may call What this function cannot/should not do Set to bytes4(0) Access level onlyOwner
WalletFactory.sol	addApprovedToken(token) / removeApprovedToken(token)	What this function can do Add/remove a token to/from the Auto-Sweep approve allowlist Main invariant(s) Array and boolean mapping stay in sync (swap-and-pop on remove) Access level onlyOwner
WalletFactory.sol	setAutoSweepConfig(key, pool, selector, tokens[])	What this function can do Batch-replace key, pool, selector, and the entire approved-token set What this function cannot/should not do Accept address(0) for key/pool or bytes4(0) selector Access level onlyOwner
WalletFactory.sol	getDefaultAutoSweepKey() / getDefaultPoolContract() / getDepositSelector() / getApprovedTokens() / isTokenApproved(token)	What this function can do Read-only Auto-Sweep config Access level onlyOwner



Contract Name	Function	Category
WalletFactory.sol	upgradeToAndCall(newImpl, data)	What this function can do Upgrade the Factory logic Access level onlyOwner (via _authorizeUpgrade)
WalletFactory.sol	transferOwnership / acceptOwnership / renounceOwnership (Ownable2Step)	What this function can do Two-step ownership handover; renounce permanently voids the Factory's admin surface What this function cannot/should not do Renouncing with pending upgrades or unset defaults bricks the system Access level onlyOwner / pending-owner



Paymaster.sol — ERC-4337 paymaster, tiered sponsorship (owner-controlled, UUPS)

Contract Name	Function	Category
Paymaster.sol	initialize(entryPoint, owner)	What this function can do Bind Paymaster to an EntryPoint and owner; seed whitelisted selectors (ERC-20 transfer/approve, SmartWallet execute/executeBatch, owner/session/recovery management) What this function cannot/should not do Run twice Main invariant(s) Seeded selector whitelist is a known-safe baseline Access level initializer
Paymaster.sol	validatePaymasterUserOp(userOp, userOpHash, maxCost)	What this function can do Approve or reject gas sponsorship based on initCode-present (counterfactual tier), selector whitelist (deployed tier), and target whitelist (for execute/executeBatch) What this function cannot/should not do Be called by any address other than EntryPoint; sponsor a call whose maxCost > MAX_GAS_COST (1 ether); read cross-contract storage during validation (ERC-7562 compliance) Main invariant(s) Deterministic on userOp; no external state reads Access level msg.sender == entryPoint enforced inline



Contract Name	Function	Category
Paymaster.sol	postOp(mode, context, actualGasCost, actualUserOpFeePerGas)	What this function can do Emit GasSponsored on settlement What this function cannot/should not do Be called by any address other than EntryPoint Access level msg.sender == entryPoint enforced inline
Paymaster.sol	deposit()	What this function can do Forward msg.value to entryPoint.depositTo(address(this)) Access level onlyOwner nonReentrant
Paymaster.sol	withdraw(to, amount)	What this function can do Transfer ETH from the Paymaster contract balance (not the EntryPoint deposit) to to What this function cannot/should not do Fail silently (reverts on low-level call failure) Access level onlyOwner nonReentrant
Paymaster.sol	withdrawFromEntryPoint(to, amount)	What this function can do Pull amount from the EntryPoint's paymaster deposit to to Access level onlyOwner nonReentrant



Contract Name	Function	Category
Paymaster.sol	setTargetWhitelist(target, whitelisted)	What this function can do Toggle execute/executeBatch target permissibility What this function cannot/should not do Self-calls (target == wallet) are always allowed without whitelist Access level onlyOwner
Paymaster.sol	setSelectorWhitelist(selector, whitelisted)	What this function can do Toggle per-selector sponsorship Access level onlyOwner
Paymaster.sol	recordDeposit(wallet, amount)	What this function can do Off-chain-driven bookkeeping for the (currently commented-out) premium-tier path Note Not automated from on-chain transfer events – owner-asserted Access level onlyOwner
Paymaster.sol	setMinDepositForFullSponsorship(amount)	What this function can do Tune the deposit threshold that (when wired) would unlock full sponsorship Access level onlyOwner
Paymaster.sol	getDeposit()	What this function can do Query the Paymaster's EntryPoint deposit balance Access level Public view



Contract Name	Function	Category
Paymaster.sol	upgradeToAndCall(newImpl, data)	What this function can do Upgrade Paymaster logic Access level onlyOwner (via _authorizeUpgrade)
Paymaster.sol	transferOwnership / acceptOwnership	What this function can do Two-step ownership handover Access level onlyOwner / pending-owner
Paymaster.sol	receive()	What this function can do Accept ETH transfers directly into the Paymaster contract balance (separate from the EntryPoint deposit) Access level Public payable

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
SmartWallet.sol (per-wallet proxy)	ETH + arbitrary ERC-20	User's native ETH (from receive) and any ERC-20 tokens held at the wallet's address; may also hold unused prefund after validateUserOp
Paymaster.sol	ETH	Paymaster contract balance (from receive()) and direct transfers) + ETH deposited into EntryPoint on behalf of address(this)
WalletFactory.sol	–	No custody. Deploys proxies deterministically; does not hold user funds



3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
SmartWallet.sol	receive()	ETH	–	Public (Anyone)	Unsolicited ETH donation is permitted; no accounting tracks senders
SmartWallet.sol	validateUserOp (prefund path)	–	ETH (to EntryPoint)	onlyEntryPoint	Pulls missingAccountFunds out of wallet balance to EntryPoint; trusted caller, but fails the UserOp if prefund transfer reverts
SmartWallet.sol	execute(target, value, data)	–	ETH + tokens (via arbitrary calldata)	onlyOwnerOrEntryPoint nonReentrant	Primary drain path – attacker-reachable only via compromised passkey, EOA backup, session key within scope, or Auto-Sweep key (bounded to approve(pool) + deposit(pool)). Non-standard ERC-20 semantics can break downstream accounting



Contract	Function	Asset In	Asset Out	Caller	Risk Notes
SmartWallet.sol	executeBatch	—	ETH + tokens	onlyOwnerOrEntryPoint nonReentrant	Same as execute, atomic on batch
SmartWallet.sol	Auto-Sweep branch of _validateEOASignatureWithPermission	—	ETH + token allowance (via approve)	EntryPoint (post-validation; execute phase)	Sweep key signs execute(pool, 0, deposit(...)) or execute(token, 0, approve(pool, X)); token allowance is bounded only by the value in the signed calldata — compromised sweep key can set approve(pool, MAX) on any approved token, then use any pool flow to pull funds
Paymaster.sol	receive()	ETH	—	Public	Accepts deposits into contract balance; does not auto-forward to EntryPoint deposit
Paymaster.sol	deposit()	ETH (msg.value)	ETH (to EntryPoint)	onlyOwner nonReentrant	Moves ETH from caller into EntryPoint paymaster deposit
Paymaster.sol	withdraw(to, amount)	—	ETH (contract balance)	onlyOwner nonReentrant	Direct drain of contract balance; owner-trusted



Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Paymaster.sol	withdrawFromEntryPoint(to, amount)	—	ETH (from EntryPoint deposit)	onlyOwner nonReentrant	Settlement drain of sponsoring funds
Paymaster.sol	postOpSettlement	—	ETH (deducted from EntryPoint deposit by EntryPoint itself)	onlyEntryPoint	Implicit outflow — EntryPoint debits Paymaster's deposit by actualGasCost; no in-contract call is required
WalletFactory.sol	createWallet / createWalletWithAutoSweep	—	—	onlyOwner nonReentrant	No value flow; deploys a proxy whose address becomes a new asset-holding entity

Closing Summary

In this report, we have considered the security of Bullbit Team. We performed our audit according to the procedure described above.

The audit identified multiple low severity issues, and several informational findings. All identified issues have been documented in this report and shared with the Bullbit team and the Bullbit team acknowledged one and resolved the remaining issues mentioned above

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers.

With eight years of expertise, we've secured over 1500 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

**8+**

Years of Expertise

1M+

Lines of Code Audited

50+

Chains Supported

1500+

Projects Secured

Follow Our Journey



AUDIT REPORT

May 2026

For



Canada, India, Singapore, UAE, UK

www.quillaudits.com

audits@quillaudits.com